

Machine Learning Intelligent Chip Design

Homework3 Implement NoC by SystemC

Description

NoC (Network-on-Chip) is a promising architecture that can help overcome communication bottlenecks and performance limitations in modern computer systems. It decouples computing resources from communication resources, allowing for large-scale parallel processing and highly flexible communication channel configurations that can be optimized based on specific application requirements. Additionally, NoC is highly fault-tolerant and scalable, providing a powerful foundation for future integrated circuit and system architectures.

Implementation Details

In HW3, you are required to implement a 4x4 mesh-based NoC architecture as shown in Figure 1. The system architecture includes the following two types of modules:

- **Router:**

The routers will be responsible for routing flits between different components within the network.

- **Core:**

Each router will be connected to a core module, which includes the Processing Element (PE) and the Network Interface (NI). The PE generates data packets, while the NI manages communication between the PE and the router.

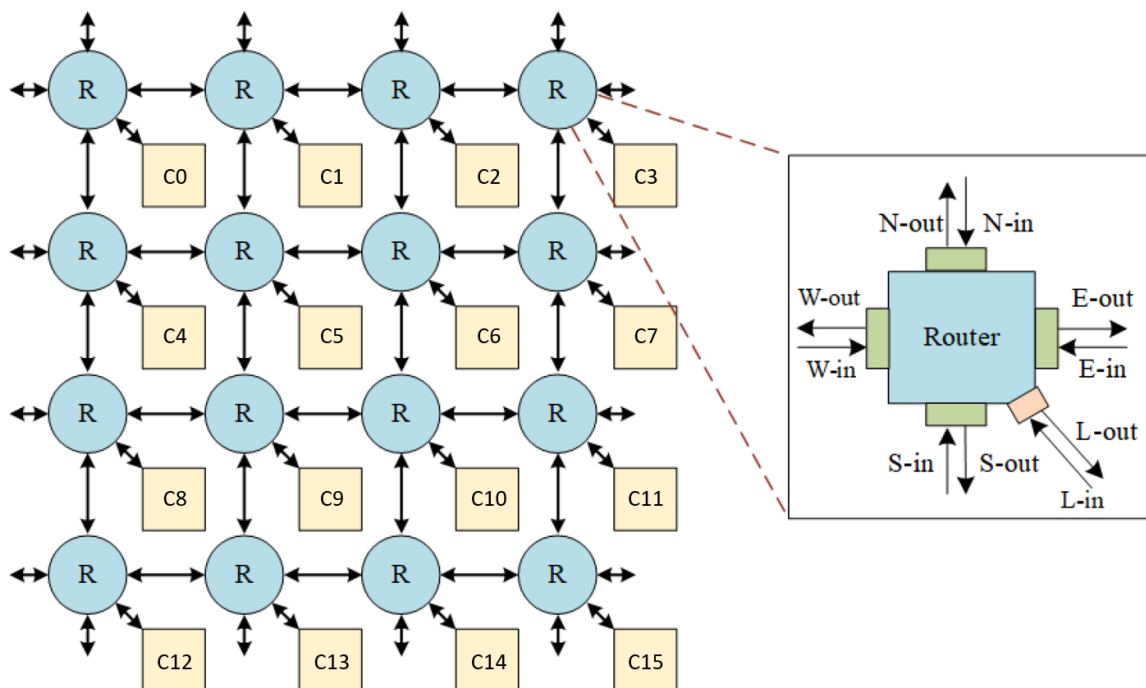


Figure 1. 4x4 mesh-based NoC architecture

To simplify the complexity of system design for this assignment, TA will provide a pre-written PE, it should be noted that **the PE cannot be modified**. The PE will be encapsulated within the core module and mainly consists of three functions:

- **void init(int pe_id)**

You need to call this function at the beginning of the simulation. The pe_id is numbered sequentially, starting from 0 in the upper-left corner, as shown in Figure 1.

- **Packet* get_packet()**

Each time you call this function, you can obtain a send packet. If the PE has no more packets to send, this function will return **NULL**. The definition of a packet is shown in Figure 2, each packet contains a **source_id** and a **dest_id**, along with a floating-point vector datas. The length of the vector in each packet is different.

```
struct Packet {
    int source_id;
    int dest_id;
    vector<float> datas;
};
```

Figure 2. Packet structure

- **void check_packet(Packet* p)**

When all flits of a packet are received, you need to pack these flits into a packet and send it to the PE by calling this function. The PE will verify whether the packet is correct. When all PEs receive the correct packets, the simulation will stop immediately and display the following screen.

```
0:46 es26mlchipTA05@ee31[~/hw3_test]$ make
g++ -I /usr/include/systemc-2.3.3 -I /usr/include/systemc-2.3.3/lib-linux64 -o run *.cpp -lsystemc-2.3.3/lib-linux64
./run

SystemC 2.3.3-Accellera --- Mar  3 2026
Copyright (c) 1996-2018 by all Contributors
ALL RIGHTS RESERVED

=====
+                                     +
+  Congratulations !!                +
+                                     +
+  Simulation completed              +
+  at 1021 th cycle                  +
+                                     +
+                                     +
=====

Info: /OSCI/SystemC: Simulation stopped by user.
0:46 es26mlchipTA05@ee31[~/hw3_test]$
```

Figure 3. Correct simulation result

Additionally, TA also provides the port definitions of Core and Router modules (Figure 4). You need to connect the core to the router, and the router to the top, bottom, left, and right routers, as shown in the architecture in Figure 1. As a reminder, since the size of each flit is limited to 34 bits, the packet should be decomposed before it is sent to the router.

```
SC_MODULE( Core ) {
    sc_in < bool > rst;
    sc_in < bool > clk;
    // receive
    sc_in < sc_lv<34> > flit_rx;
    sc_in < bool > req_rx;
    sc_out < bool > ack_rx;
    // transmit
    sc_out < sc_lv<34> > flit_tx;
    sc_out < bool > req_tx;
    sc_in < bool > ack_tx;
}

SC_MODULE( Router ) {
    sc_in < bool > rst;
    sc_in < bool > clk;

    sc_out < sc_lv<34> > out_flit[5];
    sc_out < bool > out_req[5];
    sc_in < bool > in_ack[5];

    sc_in < sc_lv<34> > in_flit[5];
    sc_in < bool > in_req[5];
    sc_out < bool > out_ack[5];
}
```

Figure 4. Port definitions of Core module and Router module

Figure 5 is an example of flit format definitions, the first two bits are used to identify the header, body or tail flit. You can reference the definition example or customize the flit format and even modify the port definition. Please explain your design considerations (such as latency, bandwidth, complexity, etc.) in detail in the report.

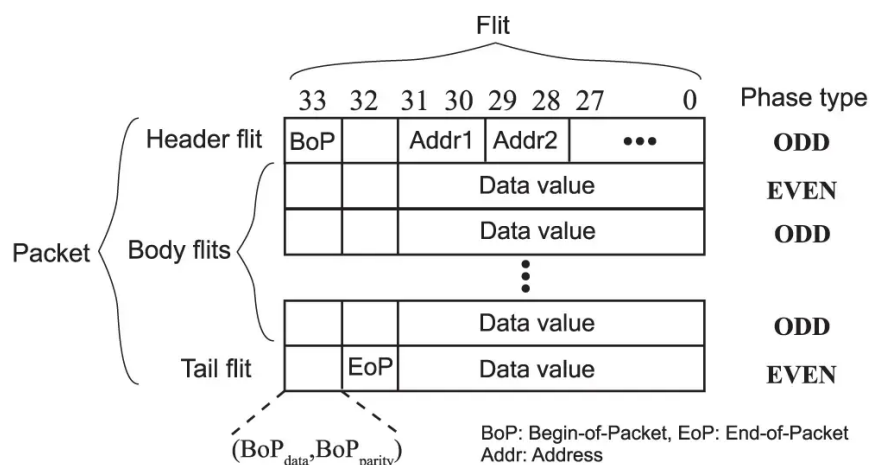


Figure 5. Example of flit format definitions

In the main function, there are three separate parts. The signals declaration, modules declaration and modules connection. You can reference Figure 4 to declare all the signal you need in the main function and interconnect these routers and cores to construct your network. For the pattern files, the data format is "TO <dest_id> <data length> <data>" or "FROM <source_id> <data length> <data>". Each PE will read the corresponding file in the pattern folder according to its id. **You are not allowed to modify pattern**, and you also don't need to process them yourself, but understanding them will help you debug.

Implement Notes

A key aspect of the implementation will be the choice of routing policy employed by the routers (e.g., XY routing, west-first adaptive routing, etc.). This policy will determine how data packets are transmitted through the network and affects simulation time.

It is important to note that in this assignment, **only `sc_in` , `sc_out` , `sc_signal` can be used for ports. Channels and interfaces that were utilized in HW2 are not allowed**. Additionally, **using pointer in port definition is also forbidden** as it doesn't make sense in hardware design.

This assignment encourages collaboration with AI. Therefore, an AI-generated file will be provided. You may use this file as a basis for optimization, or choose to rewrite the file from scratch. Please note that your optimized cycle count **must lower** the provided baseline (1021 cycles).

Submission Guidelines & Grading Policy

- **The grading breakdown for this assignment is as follows:**
 - **Report : 60%**
 - ◆ Simulation results demonstrate the predicted output for the provided input data.
 - ◆ How do you design the router and NI? What routing algorithm do you use? What is the depth of the buffer? Do you use virtual channels?
 - ◆ Based on the .ppt provided by the TA, the report must explain **how many pipeline stages** your designed router is divided into, and how the **cycle time** is set.
 - ◆ Your implementation approach, challenges faced, and any observations or insights gained during the implementation and simulation process.
 - ◆ If the code is not submitted, **no points will be awarded for the report!**
 - **Simulation Result : 40%**
 - ◆ You need to complete a NoC architecture similar to Figure 1. After executing the 'make' command, the terminal must display a 'Congratulations' message as shown in Figure 3. **TA will run your code to check result.**
 - ◆ **Note that your optimized cycle count must lower the provided baseline (1021 cycles), or you will get 0 points.**
- For the code submission, please use the **compression and submission command provided by the TA.**
- Please submit the report file to the new E3. The name of your report file is **report_mlchipXXX.pdf (XXX is your account ID)**. If the file violates the naming rule and the file format, **10 points will be deducted.**
- Ensure that your code is well-commented and organized for clarity and understanding.
- **The following are rules that must be absolutely adhered to. Any**

violation will result in 0 points.

- Plagiarism is forbidden.
- You cannot modify the provided Makefile, PE (pe.cpp 、pe.h), or pattern files.
- Any words with “error”, “fail” or “Congratulations” can’t be used in your assignment.
- You cannot use any Chinese characters in the assignment, including within comments.
- Only sc_in 、sc_out 、sc_signal can be used for ports. Channels and interfaces that were utilized in HW2 are not allowed.
- Using pointer in port definition is forbidden.

Submission & demo command

- Please use make commands to execute your SystemC code.
- Follow following command in 09_SUBMIT folder to submit your code and demo.

1. ./00_tar

2. ./01_submit

3. ./02_check